线程安全的无锁栈

Posted on 2023-12-16 14:59:41 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Zc8PJGIzR1QS2B3XvVcnek7p3y)

简介

前文我们通过锁的互斥机制实现了并发安全的栈，队列，查找表，以及链表等结构。接下来本文介绍通过无锁的原子变量的方式实现对应的容器，我们这一篇先从无锁的方式实现栈讲起。

栈的设计思路

栈容器是一种先进后出的结构，简单来讲，我们将n个元素1，2，3，4依次入栈，那么出栈的顺序是4，3，2，1.

先考虑单线程情况下操作顺序

1 创建新节点

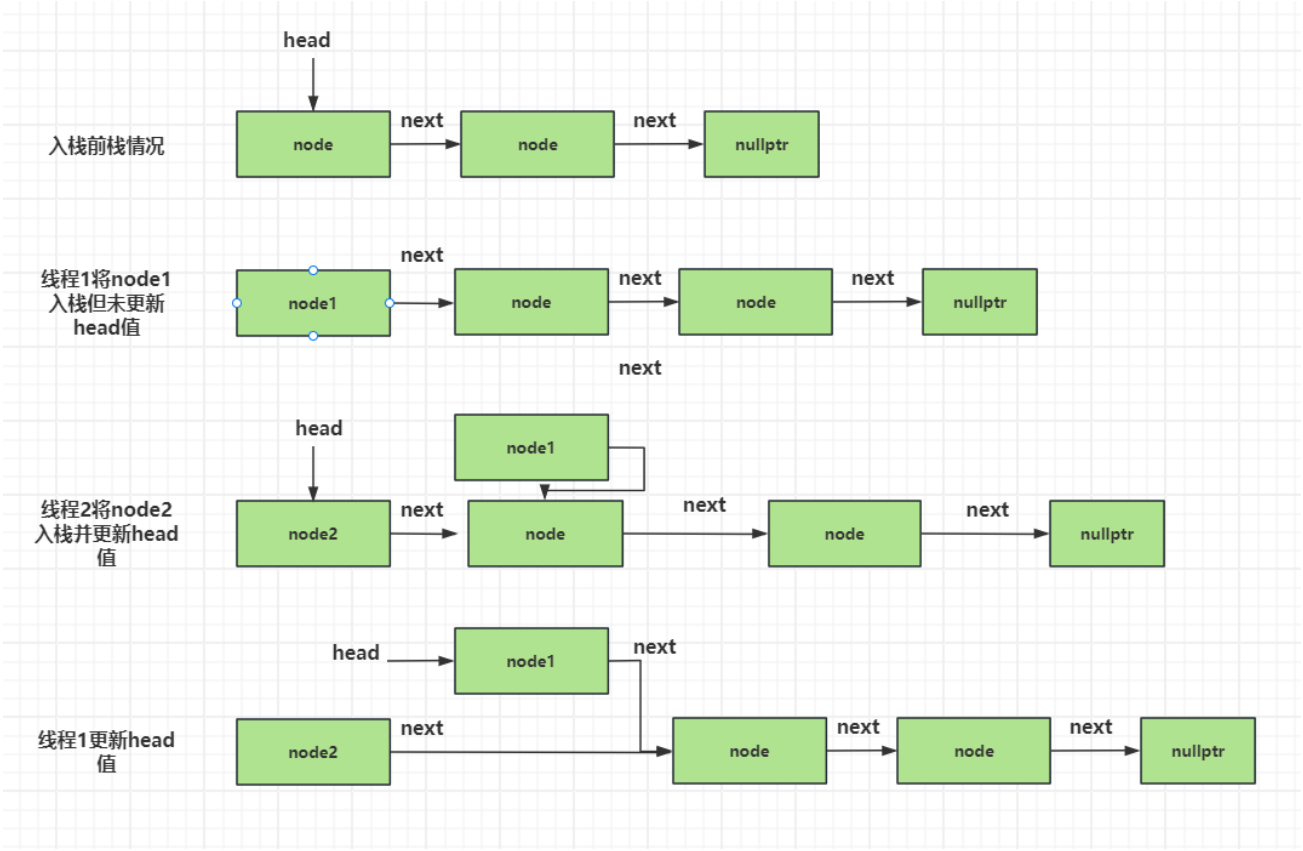
2 将元素入栈，将新节点的next指针指向现在的head节点。

3 将head节点更新为新节点的值。

再考虑多线程的情况下

假设线程1执行到第2步，没来得及更新head节点的值为新节点的值。此时线程2也执行完第2步，将head更新为线程2插入的新节点，之后线程1又将head更新为线程1的新插入节点，那么此时head的位置就是错的。

如下图



我们可以通过原子变量的compare_exchange(比较交换操作)来控制更新head节点，以此来达到线程安全的目的。

我们先定义节点的结构

```
1. template<typename T>
2. struct node
3. {
4.     T data;
5.     node* next;
6.     node(T const& data_) :
7.         data(data_)
8.     {}
9. };
```

一个node节点包含两部分内容，一个T类型的数据域，一个 `node*` 的next指针，指向下一个节点。

我们接下来定义一个无锁栈的结构

```
template<typename T>
class lock_free_stack
{
private:
    lock_free_stack(const lock_free_stack&) = delete;
    lock_free_stack& operator = (const lock_free_stack&) = delete;

    std::atomic<node*> head;
public:
    lock_free_stack() {}
}
```

我们同样将拷贝构造和拷贝赋值删除了，将head设置为原子变量，这样我们实现push操作的时候，可以通过比较交换的方式达到安全更新head的效果。

```
template<typename T>
void push(const T& value){
    auto new_node = new Node(value)
    do{
        new_node->next = head.load();
    }while(!head.compare_exchange_strong(new_node->next, new_node));
}
```

当然<Concurrency Programing C++>书中的做法更简略一些

```
template<typename T>
void push(const T& value){
    auto new_node = new Node(value)
    do{
        new_node->next = head.load();
    }while(!head.compare_exchange_weak(new_node->next, new_node));
}
```

我还是建议大家用do-while的方式实现，这样我们可以在do-while中增加很多自己的定制逻辑,另外推荐大家用compare_exchange_weak，尽管存在失败的情况，但是他的开销小，所以compare_exchange_weak返回false我们再次重试即可。

单线程情况下pop操作的顺序

1 取出头节点元素

2 更新head为下一个节点。

3 返回取出头节点元素的数据域。

多线程情况下，第1，2点同样存在线程安全问题。此外我们返回节点数据域时会进行拷贝赋值，如果出现异常会造成数据丢失，这一点也要考虑。
所以我们同样通过head原子变量比较和交换的方式检测并取出头部节点。

我们先写一个单线程版本

```
1. template<typename T>
2. void pop(T& value){
3.     node* old_head = head.load(); //1
4.     head = head->next; //2
5.     value = old_head->data;
6. }
```

我们知道1处和2处在多线程情况下会存在线程安全问题。所以我们用原子变量的比较交换操作改写上面的代码

```
1. template<typename T>
2. void pop(T& value){
3.     do{
4.         node* old_head = head.load(); //1
5.     }while(!head.compare_exchange_weak(old_head, old_head->next)); //2
6.     value = old_head->data; //3
7. }
```

我们通过判断head和old_head的值是否相等，如果相等则将head的值设置为old_head的下一个节点，否则返回false，并且将old_head更新为当前head的值(比较交换函数帮我们做的)。

我们看上面的代码，有三点严重问题

1 未判断空栈的情况，这一点比较好处理，如果为空栈我们可以令pop返回false，或者抛出异常，当然抛出异常不可取。

2 将数据域赋值给引用类型的value时存在拷贝赋值(3处)，我们都知道拷贝赋值会存在异常的情况，当异常发生时元素已经从栈定移除了，破坏了栈的结构，这一点和锁处理时不一样，锁处理的时候是先将元素数据域取出赋值再出栈，所以不会有问题，但是无锁的方式就会出现栈被破坏的情况。解决方式也比较简单，数据域不再存储T类型数据，而是存

储 `std::shared_ptr<T>` 类型的数据。智能指针在赋值的时候不会产生异常。

3 未释放弹出的节点的内存。

那我们修改之后的代码就是这样了

```
1. class lock_free_stack
2. {
3. private:
4.     struct node
5.     {
6.         std::shared_ptr<T> data;
7.         node* next;
8.         node(T const& data_) : //<--- - 1
9.             data(std::make_shared<T>(data_))
10.        {}
11.    };
12.    lock_free_stack(const lock_free_stack&) = delete;
13.    lock_free_stack& operator = (const lock_free_stack&) = delete;
14.    std::atomic<node*> head;
15. public:
16.    lock_free_stack() {}
17.    void push(T const& data)
18.    {
19.        node* const new_node = new node(data);    //<--- - 2
20.            new_node->next = head.load();    //<--- - 3
21.            while (!head.compare_exchange_weak(new_node->next, new_node));    //<--- - 4
22.    }
23.
24.    std::shared_ptr<T> pop() {
25.        node* old_head = nullptr; //1
26.        do {
27.            old_head = head.load(); //2
28.            if (old_head == nullptr) {
29.                return nullptr;
30.            }
31.        } while (!head.compare_exchange_weak(old_head, old_head->next)); //3
32.
33.        return old_head->data;  //4
34.    }
35. };
```

简单描述下pop函数的功能，

1 处初始化一个临时old_head的变量，

2 处加载head节点

3 处通过比较和交换操作，判断head和old_head是否相等，如相等则将head更新为old_head的next节点。如不相等，将old_head更新为head的值(compare_exchange_weak自动帮我们做了),再次进入循环。尽管2处又加载了一次head的值给old_head有些重复，但是为了代码的可读性和指针判空，我觉得这么写更合适一点。

资源回收的问题我们还没处理。
我们先实现一个简单的回收处理逻辑

```
1. template<typename T>
2. std::shared_ptr<T> pop() {
3.     node* old_head = nullptr; //1
4.     do {
5.         old_head = head.load();
6.         if (old_head == nullptr) {
7.             return nullptr;
8.         }
9.     } while (!head.compare_exchange_weak(old_head, old_head->next)); //2
10.
11.    std::shared_ptr<T> res;   //3
12.    res.swap(old_head->data); //4
13.    delete old_head;  //5
14.    return res;  //6
15. }
```

上面的代码在3处定义了一个T类型的智能指针res用来返回pop的结果，所以在4处将old_head的data值转移给res，这样就相当于清除old_head的data了。

在5处删除了old_head. 意在回收数据，但这存在很大问题，比如线程1执行到5处删除old_head，而线程2刚好执行到2处用到了和线程1相同的old_head，线程2执行compare_exchange_weak的时候 `old_head->next` 会引发崩溃。

所以要引入一个机制，延迟删除节点。将本该及时删除的节点放入待珊节点。基本思路如下

1 如果有多个线程同时pop，而且存在一个线程1已经交换取出head数据并更新了head值，另一个线程2即将基于旧有的head获取next数据，如果线程1删除了旧有head，线程2就有可能

产生崩溃。这种情况我们就要将线程1取出的head放入待删除的列表。

2 同一时刻仅有一个线程1执行pop函数，不存在其他线程。那么线程1可以将旧head删除，并删除待删列表中的其他节点。

3 如果线程1已经将head节点交换弹出，线程2还未执行pop操作，当线程1准备将head删除时发现此时线程2进入执行pop操作，那么线程1能将旧head删除，因为线程2读取的head和线程1不同(线程2读取的是线程1交换后新的head值)。此情形和情形1略有不同，情形1是两个线程同时pop只有一个线程交换成功的情况，情形3是一个线程已经将head交换出，准备删除之前发现线程2执行pop进入，所以这种情况下线程1可将head删除，但是线程1不能将待删除列表删除，因为有其他线程可能会用到待删除列表中的节点。

我们思考这种情形

线程1 执行pop已经将head换出

线程2 执行pop函数，发现线程1正在pop操作，线程2就将待删除的节点head(此head非线程1head)放入待删列表.

线程3 和线程2几乎同时执行pop函数但是还未执行head的交换操作，此head和线程2的head相同。

这种情况下线程1可能读取待删列表为空，因为线程2可能还未更新，也可能读取待删列表不为空(线程2已更新)，但是线程1不能删除这个待删列表，因为线程3可能在用。

那基于上述三点，我们可以简单理解为

1 如果head已经被更新，且旧head不会被其他线程引用，那旧head就可以被删除。否则放入待删列表。

2 如果仅有一个线程执行pop操作，那么待删列表可以被删除，如果有多个线程执行pop操作，那么待删列表不可被删除。

我们需要用一个原子变量threads_in_pop记录有几个线程执行pop操作。在pop结束后再减少threads_in_pop。
我们需要一个原子变量to_be_deleted记录待删列表的首节点。

那么我们先实现一个改造版本

```
std::shared_ptr<T> pop() {
    //1 计数器首先自增，然后才执行其他操作
    ++threads_in_pop;
    node* old_head = nullptr;
    do {
        //2 加载head节点给旧head存储
        old_head = head.load();
        if (old_head == nullptr) {
            --threads_in_pop;
            return nullptr;
        }
    } while (!head.compare_exchange_weak(old_head, old_head->next)); // 3
    //3处 比较更新head为旧head的下一个节点

    std::shared_ptr<T> res;
    if (old_head)
    {
        // 4 只要有可能，就回收已删除的节点数据
        res.swap(old_head->data);
    }
    // 5 从节点提取数据，而非复制指针
    try_reclaim(old_head);
    return res;
}
```

1 在1处我们对原子变量threads_in_pop增加以表示线程执行pop函数。

2 在2处我们将head数据load给old_head。如果old_head为空则直接返回。

3 3处通过head和old_head作比较，如果相等则交换，否则重新do while循环。这么做的目的是为了防止多线程访问，保证只有一个线程将head更新为old_head的下一个节点。

4 将old_head的数据data交换给res。

5 try_reclaim函数就是删除old_head或者将其放入待删列表，以及判断是否删除待删列表。

接下来我们实现try_reclaim函数

```
1. void try_reclaim(node* old_head)
2. {
3.     //1 原子变量判断仅有一个线程进入
4.     if(threads_in_pop == 1)
5.     {
6.         //2 当前线程把待删列表取出
7.         node* nodes_to_delete = to_be_deleted.exchange(nullptr);
8.         //3 更新原子变量获取准确状态，判断pop是否仅仅正被当前线程唯一调用
9.         if(!--threads_in_pop)
10.        {
11.            //4 如果唯一调用则将待删列表删除
12.            delete_nodes(nodes_to_delete);
13.        }else if(nodes_to_delete)
14.        {
15.            //5 如果pop还有其他线程调用且待删列表不为空，
16.            //则将待删列表首节点更新给to_be_deleted
17.            chain_pending_nodes(nodes_to_delete);
18.        }
19.        delete old_head;
20.    }
21.    else {
22.        //多个线程pop竞争head节点，此时不能删除old_head
23.        //将其放入待删列表
24.        chain_pending_node(old_head);
25.        --threads_in_pop;
26.    }
27. }
```

1 1处我们判断pop的线程数是否为1，并没有采用load，也就是即便判断的时候其他线程也可以pop，这样不影响效率，即便模糊判断threads_in_pop为1，同一时刻threads_in_pop可能会增加也没关系，threads_in_pop为1仅表示当前时刻走入1处逻辑之前仅有该线程执行pop，那说明没有其他线程竞争head，head已经被更新为新的值，其他线程之后pop读取的head和我们要删除的old_head不是同一个，就是可以被直接删除的。

2 处我们将当前待删除的列表交换给本线程的nodes_to_delete临时变量，表示接管待删除列表。但是能否删除还要判断是不是仅有本线程在执行pop。

3 处更新原子变量获取准确状态，判断pop是否仅仅正被当前线程唯一调用，如果是被唯一调用则删除待删列表，否则将nodes_to_delete临时变量再更新回待删列表。(因为可能有多个线程会用待删列表中的节点)

接下来我们实现delete_nodes函数, 该函数用来删除以nodes为首节点的链表，该函数写成了static函数，也可以用普通函数。

```
1. static void delete_nodes(node* nodes)
2. {
3.     while (nodes)
4.     {
5.         node* next = nodes->next;
6.         delete nodes;
7.         nodes = next;
8.     }
9. }
```

接下来实现chain_pending_node函数，该函数用来将单个节点放入待删列表

```
1. void chain_pending_node(node* n)
2. {
3.     chain_pending_nodes(n, n);
4. }
```

chain_pending_nodes接受两个参数，分别为链表的头和尾。

```
1. void chain_pending_nodes(node* first, node* last)
2. {
3.     //1 先将last的next 节点更新为待删列表的首节点
4.     last->next = to_be_deleted;
5.     //2  借循环保证 last->next指向正确
6.     //  将待删列表的首节点更新为first 节点
7.     while (!to_be_deleted.compare_exchange_weak(
8.         last->next, first));
9. }
```

1 处将last->next的值更新为to_be_deleted, 这么做的一个好处是如果有其他线程修改了to_be_deleted.能保证当前线程的last->next指向的是最后修改的to_be_deleted，达到链接待删列表的作用。

2 处可能更新失败，因为其他线程修改了to_be_deleted的值，但是不要紧，我们再次循环直到匹配last->next的值为to_be_deleted为止，将to_be_deleted更新为first的值。

接下来我们还要实现将nodes_to_delete为首的链表还原到待删列表中, 函数为

chain_pending_nodes接受一个参数为待还原的链表的首节点

```
1. void chain_pending_nodes(node* nodes)
2. {
3.     node* last = nodes;
4.     //1 沿着next指针前进到链表末端
5.     while (node* const next = last->next)
6.     {
7.         last = next;
8.     }
9.     //2 将链表放入待删链表中
10.    chain_pending_nodes(nodes, last);
11. }
```

分析

上面的无锁栈存在一个问题，就是当多个线程pop时将要删除的节点放入待删列表中，如果每次pop都会被多个线程调用，则要删除的节点就会一直往待删除列表中增加，导致待删除列表无法被回收。这个问题我们可以考虑当pop执行结束时最后一个线程回收待删列表。留作下一节分析。

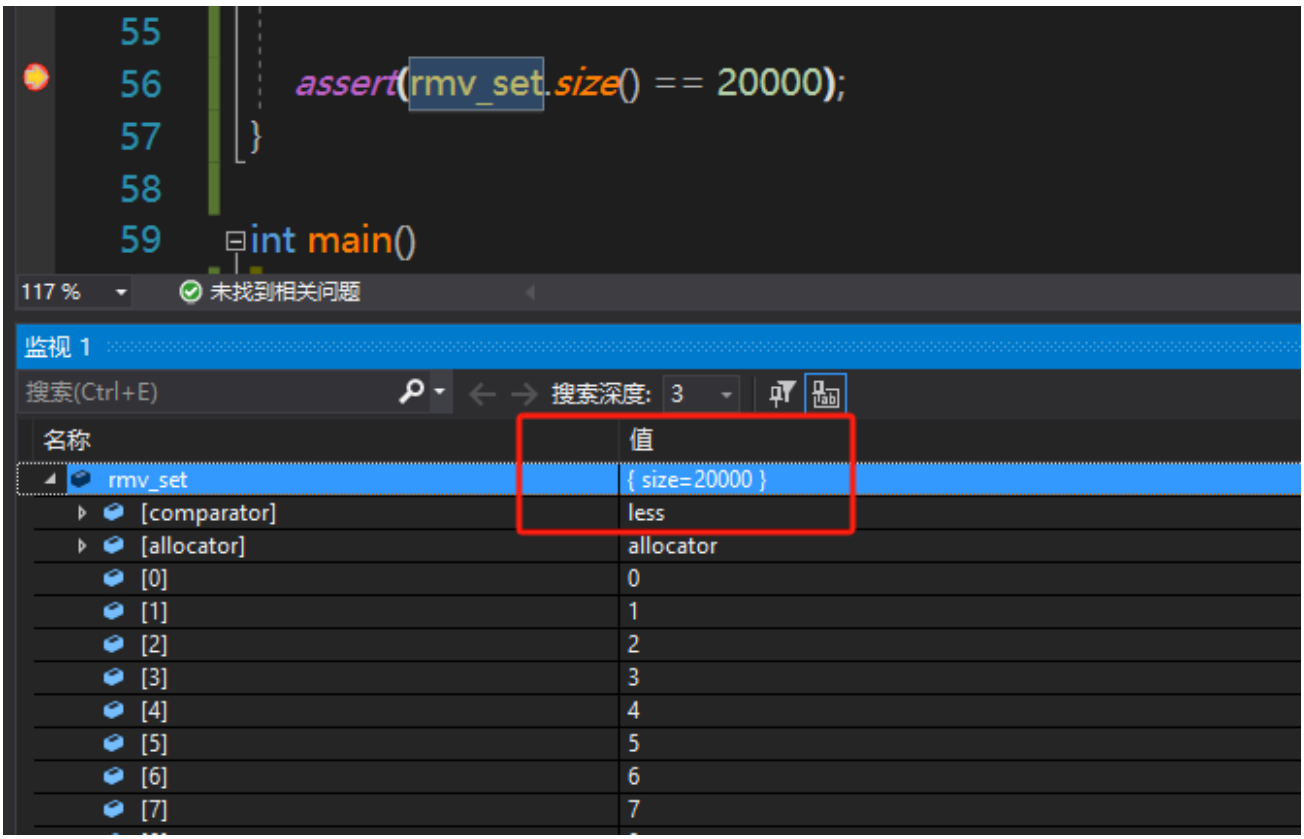
我们先写一个函数测试以下

```
1. void TestLockFreeStack() {
2.
3.     lock_free_stack<int> lk_free_stack;
4.     std::set<int>  rmv_set;
5.     std::mutex set_mtx;
6.
7.     std::thread t1([&]() {
8.         for (int i = 0; i < 20000; i++) {
9.             lk_free_stack.push(i);
10.            std::cout << "push data " << i << " success!" << std::endl;
11.            }
12.        });
13.
14.    std::thread t2([&]() {
15.        for (int i = 0; i < 10000;) {
16.            auto head = lk_free_stack.pop();
17.            if (!head) {
18.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
19.                continue;
20.            }
21.            std::lock_guard<std::mutex> lock(set_mtx);
22.            rmv_set.insert(*head);
23.            std::cout << "pop data " << *head << " success!" << std::endl;
24.            i++;
25.        }
26.     });
27.
28.    std::thread t3([&]() {
29.        for (int i = 0; i < 10000;) {
30.            auto head = lk_free_stack.pop();
31.            if (!head) {
32.                std::this_thread::sleep_for(std::chrono::milliseconds(10));
33.                continue;
34.            }
35.            std::lock_guard<std::mutex> lock(set_mtx);
36.            rmv_set.insert(*head);
37.            std::cout << "pop data " << *head << " success!" << std::endl;
38.            i++;
39.        }
40.        });
41.
42.    t1.join();
```

```
43.     t2.join();
44.     t3.join();
45.
46.     assert(rmv_set.size() == 20000);
47. }
```

1 线程t1将0到20000个数放入集合中。
2 线程t2和t3分别出栈10000次。
3 最后我们断言集合的大小为20000.

测试结果如下



可以看到我们的集合大小为20000，且数据唯一。

总结

源码链接:

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day17-LockFreeStack (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day17-LockFreeStack)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(2389) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(511) (JavaScript:void(0);)

热门评论

请输入正文

评论

gongxl1227

2023-12-20 20:46:15

找到夯住的原因了，当t1 push 结束后，栈为空时 t2和t3 没有对i++，陷入了死循环。

♡ 喜欢(1) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看

gongxl1227

2023-12-20 20:12:08

1. `std::thread t1([&]()` 中for循环的次数取2000， t2和t3中循环次数分别取10000时，程序会夯死

♡ 喜欢(1) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看

没有更多评论了,点击收起 (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)